

Aula 09 — Busca por ID em lista de objetos

Resumo

Problema: `get(indice)` é fragil porque a posição na lista pode mudar (reordenação, remoção), mas a identidade da conta (ID) não muda.

Conceitos centrais

- `ArrayList<Conta>`: lista dinâmica, tipo genérico entre `<>` restringe o que pode ser armazenado
- `import java.util.ArrayList`: necessário porque a classe não é carregada automaticamente
- Diferença array vs `ArrayList`: tamanho fixo vs dinâmico (`add`, `get`, `size`, `isEmpty`, `remove`)
- `buscarPorId`: percorre a lista com `for-each`, compara `conta.id`, retorna `Conta` ou `null`
- Contrato de retorno `null`: significa "não encontrado", exige validação antes de operar (evita `NullPointerException`)
- Busca sequencial: custo linear, aceitável em volumes pequenos/medios

Git e GitHub — Versionamento Básico

Git rastreia o histórico de alterações do código localmente.

- `git init` — inicia repositório
- `git add arquivo` — move mudança para staging
- `git commit -m "mensagem"` — grava snapshot no histórico
- `git status` / `git log` — checka estado e histórico
- `git branch nome` / `git checkout nome` — cria/troca linha de desenvolvimento paralela
- `git merge nome` — junta branch a outra

GitHub hospeda o repositório remotamente e viabiliza colaboração.

- `git remote add origin URL` — liga repositório local ao remoto
- `git push origin main` — envia commits locais para o GitHub
- `git pull origin main` — traz atualizações do remoto
- `git clone URL` — copia um repositório remoto
- Pull Request — mecanismo de revisão antes de integrar mudanças a branch principal

Fluxo típico: `commit` local frequente e pequeno, `push` para backup e colaboração, branch por funcionalidade, PR para revisão antes do `merge`.

Codigo Java — AppGestaoContas.java

Integra: ArrayList<Conta>, buscarPorId com retorno null, tratamento explicito de ausencia, saque via ID com contrato boolean, size(), isEmpty(), for-each.

```
import java.util.ArrayList;

class Conta {
    int id;
    String titular;
    double saldo;

    Conta(int p_id, String p_titular, double p_saldoInicial) {
        id = p_id;
        titular = p_titular;
        saldo = p_saldoInicial;
    }

    void depositar(double p_valor) {
        if (p_valor > 0) {
            saldo = saldo + p_valor;
        }
    }

    boolean sacar(double p_valor) {
        if (p_valor > 0 && p_valor <= saldo) {
            saldo = saldo - p_valor;
            return true;
        }
        return false;
    }

    @Override
    public String toString() {
        return "ID: " + id
            + " | Titular: " + titular
            + " | Saldo: " + String.format("%.2f", saldo);
    }
}

public class AppGestaoContas {

    public static void main(String[] args) {
        ArrayList<Conta> contas = new ArrayList<>();
        contas.add(new Conta(1, "Ana", 1000.00));
        contas.add(new Conta(2, "Bruno", 700.00));
        contas.add(new Conta(3, "Carla", 500.00));

        System.out.println("=== LISTA INICIAL ===");
        listarContas(contas);

        // Busca por ID (estável) em vez de get(índice) (frágil)
        Conta contaEncontrada = buscarPorId(contas, 2);
        if (contaEncontrada != null) {
            System.out.println("\nConta encontrada: " + contaEncontrada);
            contaEncontrada.depositar(100.00);
        }

        // Tratamento explicito de null: contrato exige validar ausência de resultado
        Conta contaInexistente = buscarPorId(contas, 99);
        if (contaInexistente == null) {
            System.out.println("\nConta 99 nao encontrada.");
        }

        // Saque com contrato boolean tratado nos dois ramos
        boolean saqueOk = sacarDeContaPorId(contas, 1, 150.00);
        System.out.println("\nSaque da conta 1 aprovado? " + saqueOk);

        System.out.println("\n=== LISTA APOS OPERACOES ===");
        listarContas(contas);

        System.out.println("\nTamanho da lista: " + contas.size());
        System.out.println("Lista vazia? " + contas.isEmpty());
    }
}
```

```

    }

    static Conta buscarPorId(ArrayList<Conta> p_contas, int p_id) {
        for (Conta conta : p_contas) {
            if (conta.id == p_id) {
                return conta;
            }
        }
        return null;
    }

    static boolean sacarDeContaPorId(ArrayList<Conta> p_contas, int p_id, double p_valor) {
        Conta conta = buscarPorId(p_contas, p_id);
        if (conta == null) {
            System.out.println("Conta " + p_id + " nao encontrada para saque.");
            return false;
        }
        return conta.sacar(p_valor);
    }

    static void listarContas(ArrayList<Conta> p_contas) {
        for (Conta conta : p_contas) {
            System.out.println(conta);
        }
    }
}

```